nexus-micro-web-builder 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: 12종의 소규모 모바일 웹 비즈니스 모델(청첩장, 돌잔치, 부고장, 디지털 명함 등)의 수익 구조, 디자인 전략, 영업 로드맵과 모바일 UI 시안을 한 화면에서 비교하는 기획 쇼케이스 도구.
- 목적/대상 사용자: 마이크로 웹 사업을 기획하는 내부 담당자(영업/기획). 외부 사용자용 서비스가 아닌 "기획 자료의 인터랙티브 버전"에 가깝다. premium-wedding-invitation(모델 0번 "모바일 청첩장")의 상위 기획 문서 역할.
- 출처: Google Vertex AI Studio 앱 export 구조(backend 프록시 + frontend Vite)이며, premium-wedding-invitation, webcraft-agency-simulator 와 backend/server.js, vertex-ai-proxy-interceptor.js, README.md 가 바이트 단위로 동일하다(PROXY_HEADER 값만 다름).

기술 스택

영역	기술	비고
프론트	React 19.2.5, TypeScript, Vite 8.0.10, Tailwind CSS(CDN 런타임)	package.json 은 전부 `latest` 지정
UI	lucide-react, recharts 3.8, clsx + tailwind-merge	
백엔드	Node.js, Express 5, google-auth-library, ws, express-rate-limit	Vertex AI 프록시 템플릿 그대로
DB	없음 (services/supabaseMock.ts 는 미사용 모킹)	
인프라	npm workspaces(frontend, backend), concurrently	배포 설정 없음
AI	Vertex AI 프록시(backend) 존재하나 프론트에서 호출하는 코드가 전혀 없음	사실상 AI 미사용

현재 완성도 추정: 약 55%

- 근거(완성): App.tsx -> Sidebar -> MainContent -> MobileMockup 흐름이 동작하고 `vite build` 성공(번들 590kB). 12개 모델 데이터(data/models.ts 174줄)와 12개 모바일 목업(components/MobileMockup.tsx 385줄)이 모두 채워져 있다.
- 근거(미완): 헤더 네비게이션/CTA/"기획서 다운로드" 버튼이 모두 동작 없음(App.tsx:27-33, components/MainContent.tsx:25-27). 백엔드와 인터셉터는 호출되는 곳이 없고, pages/, contexts/, services/, components/ui/, components/layout/ 는 App 에서 import 되지 않는 다른 템플릿("Nexus Platform" shadcn 대시보드)의 잔재다.

핵심 기능 목록

기능	상태
12개 비즈니스 모델 목록 + 선택 전환	동작
모델별 수익/디자인/영업 벤토 카드	동작
시장 분석 산점도(recharts ScatterChart)	동작(임의 스코어)
모델별 모바일 UI 목업 12종	동작(picsum 랜덤 이미지)
기획서 다운로드, 템플릿 마켓, 로그인/대시보드	미구현 또는 미연결
Vertex AI 연동	코드 없음

2. 구조 분석

경로	역할	비고
package.json	루트 workspace, npm run dev 로 프론트+백엔드 동시 실행	concurrently latest
backend/server.js (453줄)	Vertex AI REST/WS 프록시, ADC 인증, rate limit	Google 템플릿 원본, 미사용
backend/.env.local	GCP 프로젝트/리전/PROXY_HEADER	루트 .gitignore 로 제외됨(정상)
frontend/index.tsx	엔트리, 인터셉터 import	
frontend/App.tsx (48줄)	헤더 + Sidebar + MainContent, selectedId 상태	실제 앱 루트
frontend/components/Sidebar.tsx (115줄)	모델 목록 + 산점도	사용 중
frontend/components/MainContent.tsx (143줄)	벤토 카드 + 목업 배치	사용 중
frontend/components/MobileMockup.tsx (385줄)	switch 12 케이스 하드코딩 목업	사용 중, 거대 컴포넌트
frontend/data/models.ts (174줄)	12개 모델 데이터	사용 중
frontend/types.ts (17줄)	BusinessModel 타입만 정의	User/Project 타입 없음
frontend/pages/Landing,Login,Dashboard.tsx	shadcn 스타일 대시보드 템플릿	고아 코드, react-router-dom 의존
frontend/contexts/AuthContext.tsx, services/supabaseMock.ts	모킹 인증	고아 코드
frontend/components/layout/, components/ui/	대시보드 레이아웃, Button/Card/Input/Label	고아 코드
frontend/components/Calendar.tsx	0 바이트 빈 파일	삭제 대상
frontend/vertex-ai-proxy-interceptor.js	fetch/WebSocket 가로채기 shim	호출 대상 없음

데이터 흐름: 모든 데이터는 data/models.ts 의 정적 배열이다. App 이 selectedId 를 들고 Sidebar(선택) 와 MainContent(표시) 에 내려주며, MainContent 가 MobileMockup 에 modelId 를 넘겨 switch 로 12개 JSX 중 하나를 그린다. 서버 통신, 상태 영속화, 라우팅은 없다. 백엔드는 기동 시 GCP ADC 와 PROXY_HEADER 가 없으면 즉시 종료(backend/server.js:22-30)하므로, npm run dev 를 쓰려면 아무 쓸모 없는 GCP 인증을 먼저 해야 하는 구조다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	구조	backend/server.js:1-453, frontend/vertex-ai-proxy-interceptor.js:1-155	프론트에 Vertex AI 호출 코드가 하나도 없는데(grep genai/generateContent 0건) 백엔드 프록시와 인터셉터가 통째로 존재. 개발 실행에 GCP ADC 가 필수가 되어 온보딩 비용만 발생
2	높음	버그	frontend/contexts/AuthContext.tsx:2, frontend/pages/Dashboard.tsx:4, frontend/types.ts:1-17	User, Project, ActivityLog 를 ../types 에서 import 하지만 types.ts 에 해당 타입이 없음. tsconfig 가 없어 오류가 드러나지 않을 뿐 컴파일 불가 코드
3	높음	버그	frontend/pages/Login.tsx:2, pages/Landing.tsx:2, components/layout/Sidebar.tsx:2	react-router-dom import 하지만 frontend/package.json 과 package-lock.json 에 없음(lock 검색 0건). import 하는 순간 빌드 실패

#	심각도	분류	위치(경로:줄)	내용
4	중간	설정	backend/server.js:10, backend/package.json:13-19	import fetch from 'node-fetch' 이지만 dependencies 에 node-fetch 없음. google-auth-library 의 전이 의존성(lock 3.3.2)에 기대는 phantom dependency. 코드가 apiResponse.body.on('data') Node 스트림 API 를 쓰므로 내장 fetch 로 바꿀 수도 없음
5	중간	보안	frontend/vertex-ai-proxy-interceptor.js:116, backend/.env.local:20	PROXY_HEADER 시크릿이 프론트 소스에 하드코딩되어 번들에 포함. 백엔드가 127.0.0.1 바인딩이라 당장 위험은 낮지만 "시크릿" 전제가 무의미
6	중간	보안	backend/server.js:232	headers: {...apiHeaders, ...headers} 로 클라이언트가 보낸 헤더가 Authorization, X-Goog-User-Project 를 덮어쓸 수 있음(템플릿 공통)
7	중간	설정	frontend/package.json:9-19, package.json:10	모든 의존성 latest. lock 에는 react 19.2.5 / vite 8.0.10 고정이지만 lock 삭제 후 재설치 시 메이저 변경 위험. @types/react, tsconfig.json 없음 -> tsc 실행 자체 불가(TS7016 확인)
8	중간	구조	frontend/components/MobileMockup.tsx:17-385	12 케이스 switch 에 각 30줄 JSX 하드코딩. models.ts 데이터와 분리되어 모델 추가 시 두 곳 수정 필요
9	중간	성능	frontend/index.html:7, 빌드 결과	Tailwind CDN 런타임(프로덕션 비권장) + 단일 번들 590kB(recharts 포함). 코드 스플리팅 없음
10	낮음	버그	frontend/index.html:8	Google Fonts 에서 Pretendard 로드 시도. Google Fonts 에 Pretendard 가 없어 404, body font-family 미적용
11	낮음	품질	frontend/components/MobileMockup.tsx:24,57,113,140,202,250,305,327,355	picsum.photos 외부 이미지. ?wedding,couple 같은 키워드 쿼리는 picsum 이 지원하지 않아 완전 랜덤 이미지가 뜸
12	낮음	품질	frontend/components/Calendar.tsx	0 바이트 빈 파일. components/Sidebar.tsx 와 components/layout/Sidebar.tsx 이름 중복
13	낮음	품질	frontend/App.tsx:27-33, components/MainContent.tsx:25-27	네비게이션 링크 href="#", "무료 시작하기", "기획서 다운로드" 버튼 모두 핸들러 없음
14	낮음	설정	frontend/vite.config.ts:10, frontend/index.html:35	사용하지 않는 process.env.API_KEY 더미 define. index.html 이 /index.js 를 참조(실제 파일은 index.tsx, Vite 가 해석해주어 동작은 하지만 혼동 유발)

심각도 높음 상세

#1 미사용 백엔드/인터셉터. 이 프로젝트의 실제 가치는 정적 데이터 기반 쇼케이스인데, Vertex AI Studio export 템플릿의 백엔드가 그대로 따라왔다. npm run dev 는 concurrently 로 백엔드를 함께 띄우고, 백엔드는 GOOGLE_CLOUD_PROJECT, PROXY_HEADER 가 없거나 ADC 로그인이 안 되어 있으면 process.exit(1) 한다. 즉 아무 AI 기능도 없는 앱을 보기 위해 gcloud 설치와 로그인이 필요하다. 백엔드 디렉토리와 인터셉터 import(index.tsx:1)를 제거하고 순수 Vite 정적 사이트로 축소하는 것이 맞다.

#2, #3 고아 템플릿 코드. pages/, contexts/, services/, components/ui/, components/layout/ 는 "Nexus Platform" 이라는 전혀 다른 SaaS 대시보드 템플릿(영문 카피, shadcn 스타일, react-router)의 잔재다. 현재 App.tsx 가 import 하지 않아 빌드가 통과하지만, 존재하지 않는 타입과 미설치 패키지를 import 하고 있어 누군가 이 파일을 연결하는 순간 깨진다. CLAUDE.md 가 이 파일들을 정식 구조로 기술(CLAUDE.md:114-126)하고 있어 다음 작업자가 오해할 가능성이 높다. 삭제하거나 CLAUDE.md 에서 "미사용" 으로 명시해야 한다.

4. 효율성 개선 제안

- 구조/아키텍처: backend/ 와 인터셉터를 제거하고 frontend 만 남긴 단일 Vite 프로젝트로 축소. MobileMockup 의 12 케이스를 `models.ts` 에 `mockup: { theme, hero, cards[] }` 같은 선언형 데이터로 옮기고 렌더러 1개로 통합하면 385줄이 100줄 내로 줄고 모델 추가가 데이터 1건 추가로 끝난다.
- 성능: Tailwind 를 빌드 타임(@tailwindcss/vite)으로 전환, recharts 를 `React.lazy` 로 분리. picsum 대신 로컬 placeholder 이미지 또는 고정 seed URL 사용.
- 개발 생산성: tsconfig.json + @types/react 추가 후 `tsc --noEmit` 를 스크립트로. 고아 파일 삭제로 타입 오류 제거. `latest` 를 lock 기준 정확한 범위(^19.2.5 등)로 고정.
- 운영/배포: 정적 사이트라 Vercel/Netlify/GitHub Pages 에 그대로 배포 가능. CLAUDE.md 의 백엔드/환경변수 섹션을 삭제하고 "정적 쇼케이스" 로 재작성.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	backend/, vertex-ai-proxy-interceptor.js, index.tsx:1 import, vite.config.ts 프록시/define 제거. 루트 package.json 을 단일 Vite 프로젝트로 정리	P0	S	Sonnet 5.5	삭제와 설정 정리 위주의 기계적 작업
2	고아 파일 삭제: pages/, contexts/, services/, components/ui/, components/layout/, components/Calendar.tsx	P0	S	Sonnet 5.5	import 그래프가 명확하여 판단 불필요
3	tsconfig.json 생성, @types/react, @types/react-dom 추가, typecheck 스크립트 추가, 의존성 버전 lock 기준 고정	P0	S	Sonnet 5.5	설정 파일 보일러플레이트
4	CLAUDE.md 를 정적 쇼케이스 기준으로 재작성(백엔드/ADC/환경변수 섹션 삭제, 미사용 파일 목록 정리)	P1	S	Sonnet 5.5	문서화
5	MobileMockup switch 12 케이스를 선언형 데이터 + 단일 렌더러로 리팩토링(models.ts 스키마 확장 포함)	P1	M	Opus 5.5	2~3개 파일에 걸친 데이터 모델 설계와 리팩토링
6	Tailwind CDN -> @tailwindcss/vite 빌드 전환, index.html 의 @layer/커스텀 CSS 를 index.css 로 이동, Pretendard 를 실제 CDN(jsdelivr) 으로 교체	P1	S	Sonnet 5.5	스타일/설정 작업
7	picsum 랜덤 이미지를 고정 seed URL 또는 /public 로컬 이미지로 교체	P1	S	Sonnet 5.5	반복적 치환
8	"기획서 다운로드" 를 현재 모델 데이터 기반 Markdown/PDF 내보내기로 구현	P2	M	Opus 5.5	명확히 정의된 중간 규모 기능
9	헤더 네비게이션 정리(미구현 메뉴 제거 또는 섹션 앵커 연결), 반응형 모바일 레이아웃 점검(Sidebar 340px 고정)	P2	S	Sonnet 5.5	UI 수정
10	recharts lazy load 및 번들 분석(rollup-plugin-visualizer) 적용	P2	S	Sonnet 5.5	설정 수준 최적화

6. 종합 평가

- 강점 3개
  1. 12개 비즈니스 모델의 가격/디자인/영업 전략 데이터가 구체적이고 한국 시장 맞춤이어서 기획 자료로서 재사용 가치가 있다(data/models.ts).
  2. 벤토 그리드 UI 와 모바일 목업의 시각 완성도가 높고 `vite build` 가 깨지지 않는다.
  3. 상태가 `selectedId` 하나로 단순하여 이해와 수정이 쉽다.
- 약점 3개
  1. 전체 파일의 절반 이상(백엔드, 인터셉터, 대시보드 템플릿)이 실제 앱과 무관한 잔재이며 일부는 컴파일 불가 상태다.
  2. 타입 검사 인프라(tsconfig, @types)가 없고 의존성이 `latest` 라 재현 가능한 빌드가 보장되지 않는다.

3. 목업이 데이터 기반이 아닌 하드코딩이어서 모델 추가/수정 비용이 높다.

- 모델별 비율: Fable 0% / Opus 20% / Sonnet 80%
- 권장 진행 순서
  1. 1단계(P0, 반나절): 백엔드·고아 코드 삭제, tsconfig/타입 추가, 버전 고정 -> 순수 정적 Vite 프로젝트로 축소.
  2. 2단계(P1): CLAUDE.md 재작성, 목업 선언형 리팩토링, Tailwind 빌드 전환, 이미지 교체.
  3. 3단계(P2): 기획서 내보내기, 네비게이션 정리, 번들 최적화.
- 최종 권고: **아카이브(축소 보존)**. 이 프로젝트는 상품이 아니라 기획 쇼케이스이며, 실제 상품화는 premium-wedding-invitation(모델 0번)이 담당한다. 1단계만 수행하여 정적 사이트로 깨끗하게 보존하고, 모델 데이터(models.ts)는 docs 로 옮겨 기획 참고 자료로 활용하는 것을 권한다. 2~3단계는 이 도구를 영업 자료로 계속 쓸 계획이 있을 때만 진행한다.
- 주 담당 모델: **Sonnet 5.5**. 남은 작업의 대부분이 삭제, 설정 정리, 문서 재작성, 스타일 전환 같은 보일러플레이트성 작업이고, 보안/데이터/결제처럼 실수 비용이 큰 영역이 없다. 유일하게 설계 판단이 필요한 목업 선언형 리팩토링(작업 5)과 내보내기 기능(작업 8)만 Opus 5.5 에 맡기면 충분하다.